UNIVERSIDAD PRIVADA DOMINGO SAVIO

Facultad de Ingeniería

ACTIVIDAD 4 · SERVICIOS DE INTERCAMBIO DE DATOS, API REST E INTERFAZ AJAX

Sistema web de la Asociación «Manos del Oriente»

Laboratorios 12, 13 y 14: JSON, XML, RSS, SOAP, API REST con tokens y consumo asíncrono con fetch

Integrantes:

1. Daniela Cayo Mamani
2. Juan Jose Vicente Cossio
3. Ronald Salvatierra Flores
4. Yordy Tolavi Ruiz
5. Alejandro Cadario Lopez

Carrera: Ingeniería de Sistemas

Asignatura: Tecnología Web II (SIS-0151)

Docente: M.Sc. Ing. Jimmy Nataniel Requena Llorentty

Santa Cruz de la Sierra, Bolivia

26 de octubre de 2026

Índice

Resumen 5
Introducción 6
Objetivos 7
Objetivo general 7
Objetivos específicos 7
Fundamentos 8
JSON y XML 8
RSS y SOAP 8
REST, HTTP y problem+json 8
AJAX con fetch 9
Metodología y herramientas 10
Implementación del Laboratorio 12 12
Catálogo JSON y XML 12
Validación de pedidos con XSD y JSON 15
RSS y comparación SOAP/REST 17
Implementación del Laboratorio 13 19
Tokens y seguridad 19
Implementación del Laboratorio 14 28
Pruebas y evidencias 33
Bitácora de IA 36
Resultados y discusión 37
Conclusiones 38
Referencias 39
Anexos 41
Anexo A. Repositorio y archivos de la actividad 41
Anexo B. Cómo reproducir los resultados 41

Índice de figuras y tablas

Tabla 1 *Herramientas utilizadas* 10
Tabla 2 *Pedidos de prueba y por qué se rechaza cada dato* 15
Tabla 3 *Comparación SOAP y REST medida en este equipo (26/10/2026)* 17
Tabla 4 *Contrato de la API /api/v1* 19
Tabla 5 *Matriz de evidencias* 33
Tabla 6 *Resumen de la bitácora de IA de la actividad* 36
Tabla 7 *Ubicación de los entregables en el repositorio* 41

Figura 1 *Versiones del entorno (E01)* 11
Figura 2 *Catálogo JSON con fecha ISO 8601 y precios como texto (E02)* 12
Figura 3 *Catálogo XML: «Bolso & cartera <tejida>» escapado por XMLWriter (E03)* 13
Figura 4 *Validación de pedidos correctos e incorrectos (E04)* 16
Figura 5 *Canal RSS 2.0 con guid y pubDate (E05)* 17
Figura 6 *SOAP frente a REST y consumo de un servicio público con cURL (E06)* 18
Figura 7 *Lista paginada con datos, meta y enlaces (E07)* 20
Figura 8 *Creación con token: 201 Created y Location (E08)* 21
Figura 9 *Reemplazo con PUT: 200 (E09)* 22
Figura 10 *DELETE: 403 para el operador, 204 para administración y 404 después (E10)* 23
Figura 11 *Errores en problem+json: 401, 401, 403, 400, 415, 422, 422, 405, 409 y 404 (E11)* . 24
Figura 12 *ETag y 304, OPTIONS con CORS, OpenAPI 3.1 y tokens guardados como SHA-256 (E12)* 26
Figura 13 *Pantalla de tokens de la API (E12)* 27
Figura 14 *Grilla actualizada con fetch al filtrar por categoría (E13)* 28
Figura 15 *Orden por precio y paginación sin recargar la página (E14)* 29
Figura 16 *Autocompletar con role=combobox, listbox y opción activa (E15)* 29
Figura 17 *Con red lenta, las búsquedas «bo» y «bord» se cancelan y gana «jean» (E16)* 30
Figura 18 *Validación en vivo: POL-001 ya existe (200) (E17)* 30
Figura 19 *Validación en vivo: POL-999 está disponible (404) (E17)* 31
Figura 20 *Cinco novedades del RSS leídas con DOMParser (E18)* 31
Figura 21 *php verificar.php: 136 de 136 (E19)* 34
Figura 22 *Pruebas en el navegador: 12 de 12 (E19)* 35

Resumen

Este informe documenta la Actividad 4 de Tecnología Web II: los laboratorios 12, 13 y 14 se resolvieron sobre el caso del docente y después se incorporaron al sistema de la Asociación de Confeccionistas Textiles «Manos del Oriente». En el laboratorio 12 se publicó el catálogo en JSON (fechas ISO 8601 y precios como texto) y en XML generado con XMLWriter, un canal RSS 2.0, un servicio SOAP comparado con su equivalente REST y la importación de pedidos validados con un esquema XSD o, en JSON, por tipos. En el laboratorio 13 se construyó la API REST /api/v1 con operaciones CRUD, autenticación con tokens Bearer guardados como hash SHA-256, errores en formato problem+json, ETag con respuesta 304, CORS y un contrato OpenAPI 3.1. En el laboratorio 14 la vitrina se actualiza con fetch sin recargar la página, cancela las peticiones viejas con AbortController, ofrece un autocompletar accesible según WAI-ARIA, lee el RSS con DOMParser y valida el código de una prenda en vivo. Ningún script inserta datos con innerHTML. Las 136 comprobaciones automáticas del proyecto pasan en MariaDB 10.11 y MySQL 8.4, las 12 pruebas en el navegador también, y las 20 evidencias de la matriz se obtuvieron de peticiones reales. La medición propia mostró que la misma prenda ocupa 1.745 bytes por SOAP y 253 bytes por REST.

Palabras clave: API REST, JSON, XML, SOAP, AJAX, fetch, OpenAPI, seguridad web.

Introducción

La asociación «Manos del Oriente» (caso verosímil declarado) reúne a socias confeccionistas de la Chiquitanía que venden en ferias, boutiques y hoteles de Santa Cruz. En las actividades anteriores el sistema pasó de un catálogo en JSON a una aplicación con MySQL, usuarios, roles, ventas y reportes. Faltaba que **otros sistemas** pudieran leer el catálogo y enviar pedidos, y que la tienda respondiera sin recargar cada página.

La guía integral de la Actividad 4 pide trabajar en secuencia 12 → 13 → 14, reutilizar la base de datos de la asociación y entregar código, API documentada, informe APA 7, bitácora de IA y repositorio en GitHub (Requena Llorentty, 2026). Este informe sigue esa estructura y presenta las evidencias numeradas E01 a E20.

Objetivos

Objetivo general

Implementar servicios de intercambio de datos, una API REST segura y una interfaz asíncrona en el sistema de la asociación, y verificar su funcionamiento con pruebas automáticas y evidencias propias.

Objetivos específicos

1. Publicar el catálogo en JSON, XML y RSS respetando tipos, escape y formatos de fecha (lab 12).
2. Validar pedidos de compradores en XML con un esquema XSD y en JSON por tipos y rangos (lab 12).
3. Comparar SOAP y REST con mediciones propias de tamaño y tiempo (lab 12).
4. Construir la API /api/v1 con CRUD, tokens, códigos HTTP correctos, problem+json, ETag y OpenAPI (lab 13).
5. Consumir la API desde la vitrina con fetch, cancelación, autocompletar accesible y validación en vivo (lab 14).
6. Documentar cada endpoint y dejar el repositorio sin secretos.

Fundamentos

JSON y XML

JSON es un formato de texto para intercambiar datos estructurados con objetos, arreglos, cadenas, números y booleanos (Bray, 2017). Como sus números son de coma flotante, los importes se envían como **texto** con dos decimales («95.00») para no perder centavos, y las fechas en ISO 8601 (International Organization for Standardization [ISO], 2019). XML representa los mismos datos con elementos y atributos; los caracteres «&» y «<» deben escaparse (World Wide Web Consortium [W3C], 2008). Un esquema XSD describe el contrato de un documento: qué elementos son obligatorios, sus tipos y sus límites (W3C, 2012).

RSS y SOAP

RSS 2.0 es un formato XML de sindicación: un canal con título, enlace y descripción, y elementos con título, guid y fecha de publicación (RSS Advisory Board, 2009). SOAP envía mensajes XML dentro de un sobre con una estructura fija (W3C, 2007); aporta un contrato formal, a cambio de mensajes más pesados.

REST, HTTP y problem+json

REST es un estilo de arquitectura basado en recursos identificados por URL y manipulados con una interfaz uniforme (Fielding, 2000). HTTP define la semántica de los métodos y de los códigos de estado: 201 con Location al crear, 204 sin cuerpo, 304 cuando el recurso no cambió según su ETag, 405 con la cabecera Allow (Fielding et al., 2022). Los errores se describen con *problem details*, un objeto JSON con type, title, status y detail (Nottingham et al., 2023). Los tokens de portador se envían en la cabecera Authorization (Jones & Hardt, 2012) y la especificación OpenAPI 3.1 describe la API de forma legible por máquinas (OpenAPI Initiative, 2021).

AJAX con fetch

La API fetch hace peticiones HTTP desde JavaScript y devuelve promesas; no rechaza la promesa ante un 404 o un 500, por lo que hay que revisar response.ok (MDN Web Docs, s. f.-b). Un AbortController cancela una petición en curso (MDN Web Docs, s. f.-a). El patrón *combobox* de WAI-ARIA define cómo anunciar una lista de sugerencias a un lector de pantalla (W3C Web Accessibility Initiative [WAI], s. f.).

Metodología y herramientas

Se trabajó en el orden de la guía: primero cada laboratorio sobre el caso del docente (carpeta laboratorios/ del repositorio), después la misma técnica en Manos del Oriente, con una prueba automática por cada requisito antes de darlo por terminado. Las evidencias se generaron con un script que hace las peticiones reales y oculta los tokens.

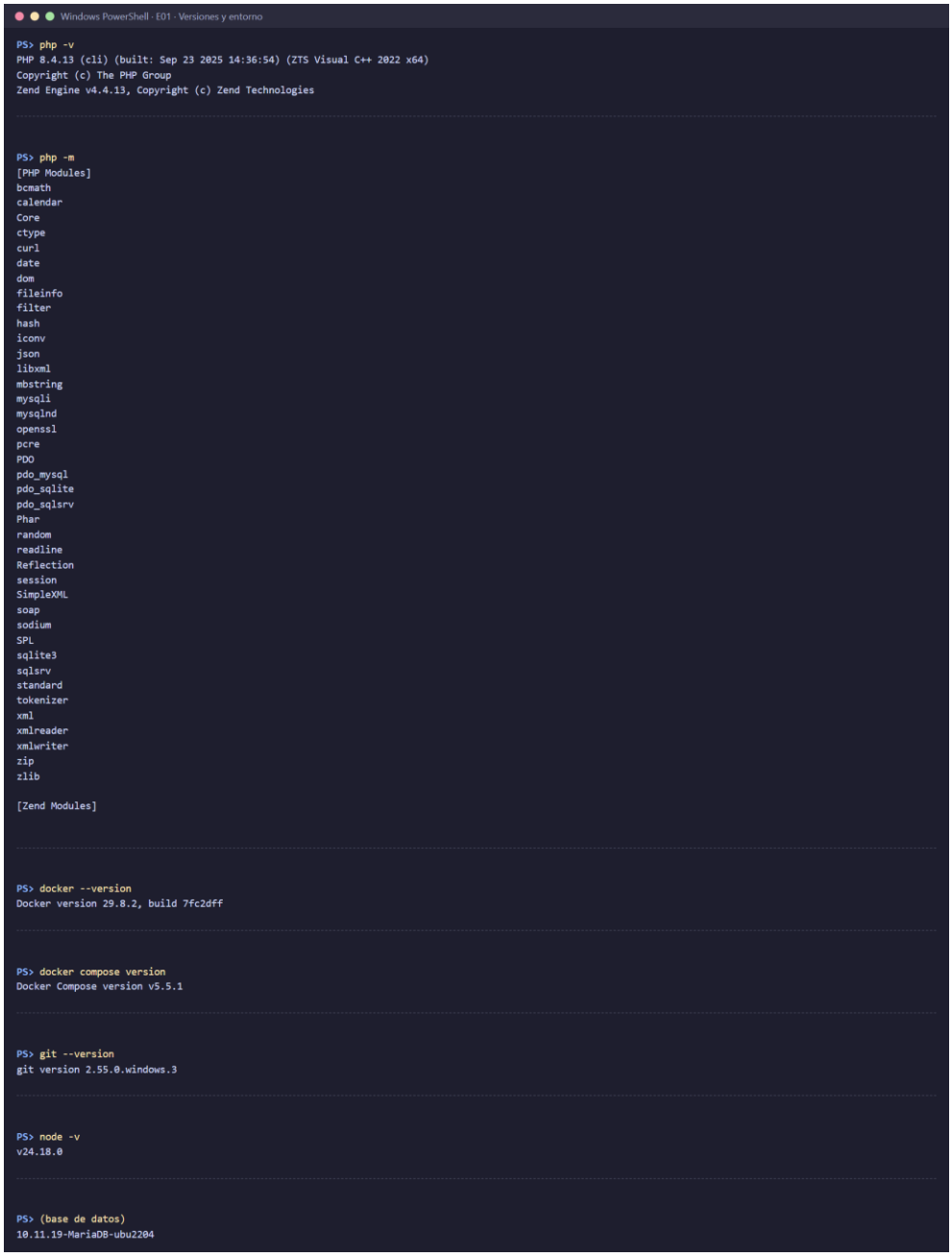
Tabla 1

Herramientas utilizadas

Herramienta	Versión	Uso
PHP	8.4.13 (local) · compatible con 8.2	Aplicación, pruebas y scripts
MariaDB / MySQL	10.11 / 8.4 (Docker)	Base de datos; las pruebas se ejecutaron en ambas
Docker Desktop	Compose v2	Bases de prueba y copia del hosting con Apache
Microsoft Edge + puppeteer-core	23.11	Pruebas en el navegador y capturas E13–E18
Git y GitHub	organización TecnologiaWeb2	Repositorio y tablero del equipo
Claude Code (Claude Opus 5.5)	—	Asistente de IA (ver la bitácora)

Figura 1

Versiones del entorno (E01)



Nota. Salida real de php -v, php -m, docker y git.

Implementación del Laboratorio 12

Catálogo JSON y XML

La función `prenda_publica()` define una sola representación para JSON, XML y SOAP: código, nombre, categoría, precio como texto, moneda, unidades libres, tallas y la socia que confeccionó la prenda. No incluye el CI ni el teléfono de las socias. `catalogo_json()` agrega la fecha de generación en ISO 8601 con el huso de Bolivia (−04:00) y `catalogo_xml()` usa XMLWriter, que escapa el texto por sí mismo (Figura 3).

Figura 2

Catálogo JSON con fecha ISO 8601 y precios como texto (E02)

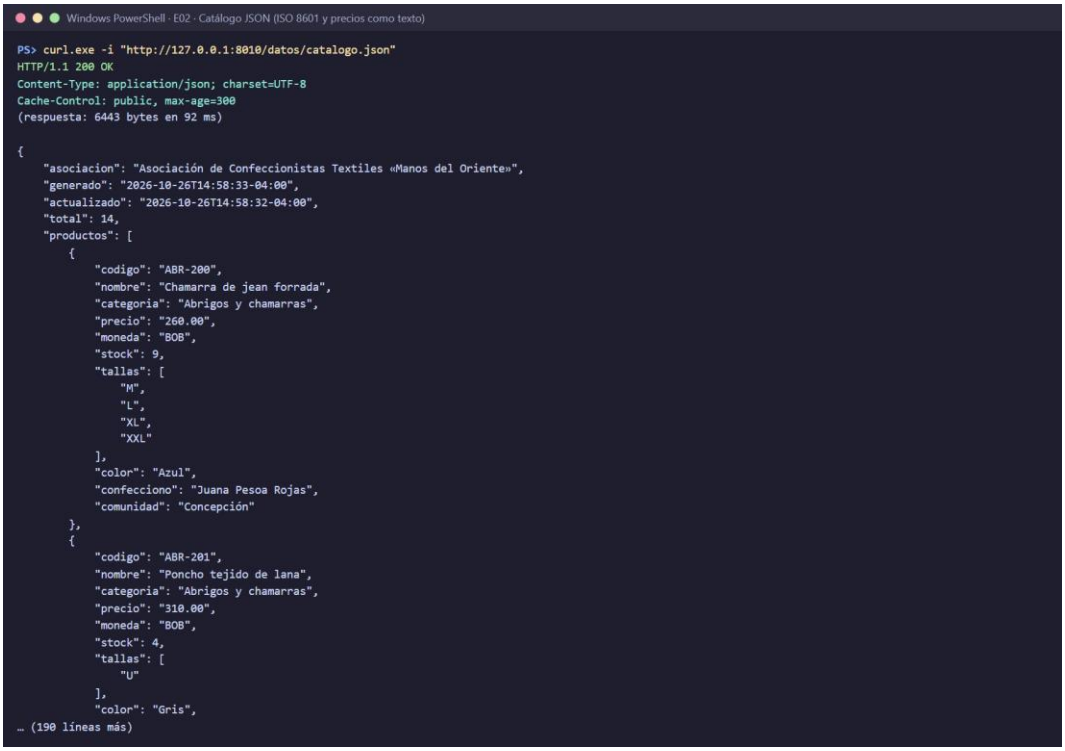
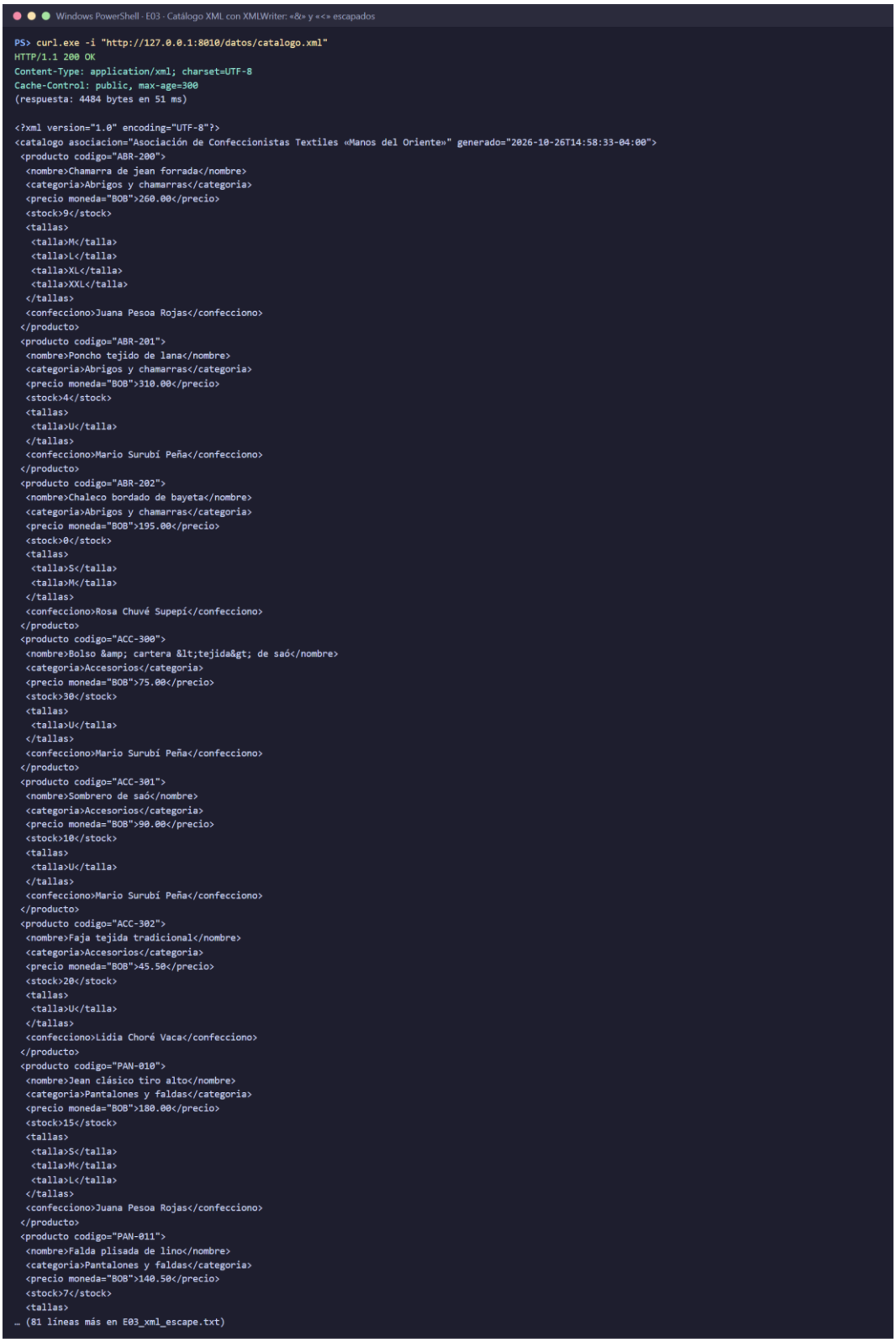


Figura 3

Catálogo XML: «Bolso & cartera <tejida>» escapado por XMLWriter (E03)



Windows PowerShell · E03 · Catálogo XML con XMLWriter: «&» y «<» escapados

```
PS> curl.exe -i "http://127.0.0.1:8010/datos/catalogo.xml"
HTTP/1.1 200 OK
Content-Type: application/xml; charset=UTF-8
Cache-Control: public, max-age=300
(respuesta: 4484 bytes en 51 ms)

<?xml version="1.0" encoding="UTF-8"?>
<catalogo asociacion="Asociación de Confeccionistas Textiles «Manos del Oriente»" generado="2026-10-26T14:58:33-04:00">
 <producto codigo="ABR-200">
  <nombre>Chamarra de jean forrada</nombre>
  <categoria>Abrigos y chamarras</categoria>
  <precio moneda="BOB">260.00</precio>
  <stock>9</stock>
  <tallas>
   <talla>M</talla>
   <talla>L</talla>
   <talla>XL</talla>
   <talla>XXL</talla>
  </tallas>
  <confecciono>Juana Pesoa Rojas</confecciono>
 </producto>
 <producto codigo="ABR-201">
  <nombre>Poncho tejido de lana</nombre>
  <categoria>Abrigos y chamarras</categoria>
  <precio moneda="BOB">310.00</precio>
  <stock>4</stock>
  <tallas>
   <talla>U</talla>
  </tallas>
  <confecciono>Mario Surubí Peña</confecciono>
 </producto>
 <producto codigo="ABR-202">
  <nombre>Chaleco bordado de bayeta</nombre>
  <categoria>Abrigos y chamarras</categoria>
  <precio moneda="BOB">195.00</precio>
  <stock>0</stock>
  <tallas>
   <talla>S</talla>
   <talla>M</talla>
  </tallas>
  <confecciono>Rosa Chuvé Supepí</confecciono>
 </producto>
 <producto codigo="ACC-300">
  <nombre>Bolso &amp;amp; cartera &amp;lt;tejida&amp;gt; de saó</nombre>
  <categoria>Accesorios</categoria>
  <precio moneda="BOB">75.00</precio>
  <stock>30</stock>
  <tallas>
   <talla>U</talla>
  </tallas>
  <confecciono>Mario Surubí Peña</confecciono>
 </producto>
 <producto codigo="ACC-301">
  <nombre>Sombrero de saó</nombre>
  <categoria>Accesorios</categoria>
  <precio moneda="BOB">90.00</precio>
  <stock>10</stock>
  <tallas>
   <talla>U</talla>
  </tallas>
  <confecciono>Mario Surubí Peña</confecciono>
 </producto>
 <producto codigo="ACC-302">
  <nombre>Faja tejida tradicional</nombre>
  <categoria>Accesorios</categoria>
  <precio moneda="BOB">45.50</precio>
  <stock>20</stock>
  <tallas>
   <talla>U</talla>
  </tallas>
  <confecciono>Lidia Choré Vaca</confecciono>
 </producto>
 <producto codigo="PAN-010">
  <nombre>Jean clásico tiro alto</nombre>
  <categoria>Pantalones y faldas</categoria>
  <precio moneda="BOB">180.00</precio>
  <stock>15</stock>
  <tallas>
   <talla>S</talla>
   <talla>M</talla>
   <talla>L</talla>
  </tallas>
  <confecciono>Juana Pesoa Rojas</confecciono>
 </producto>
 <producto codigo="PAN-011">
  <nombre>Falda plisada de lino</nombre>
  <categoria>Pantalones y faldas</categoria>
  <precio moneda="BOB">140.50</precio>
  <stock>7</stock>
  <tallas>
… (81 líneas más en E03_xml_escape.txt)
```

Validación de pedidos con XSD y JSON

Las tiendas compradoras envían pedidos en XML o JSON. El XML se carga con `LIBXML_NONET` (el analizador nunca sale a la red) y sin expandir entidades, y se valida con `schemaValidate()` contra `bd/esquemas/pedido.xsd`. El JSON se valida en código: «cantidad» debe ser un entero de 1 a 1000 («"2"» se rechaza aunque parezca un número), la fecha debe existir y los códigos deben cumplir el patrón AAA-999. Si el pedido es válido se registra con reserva de stock (módulo 2 del trabajo final).

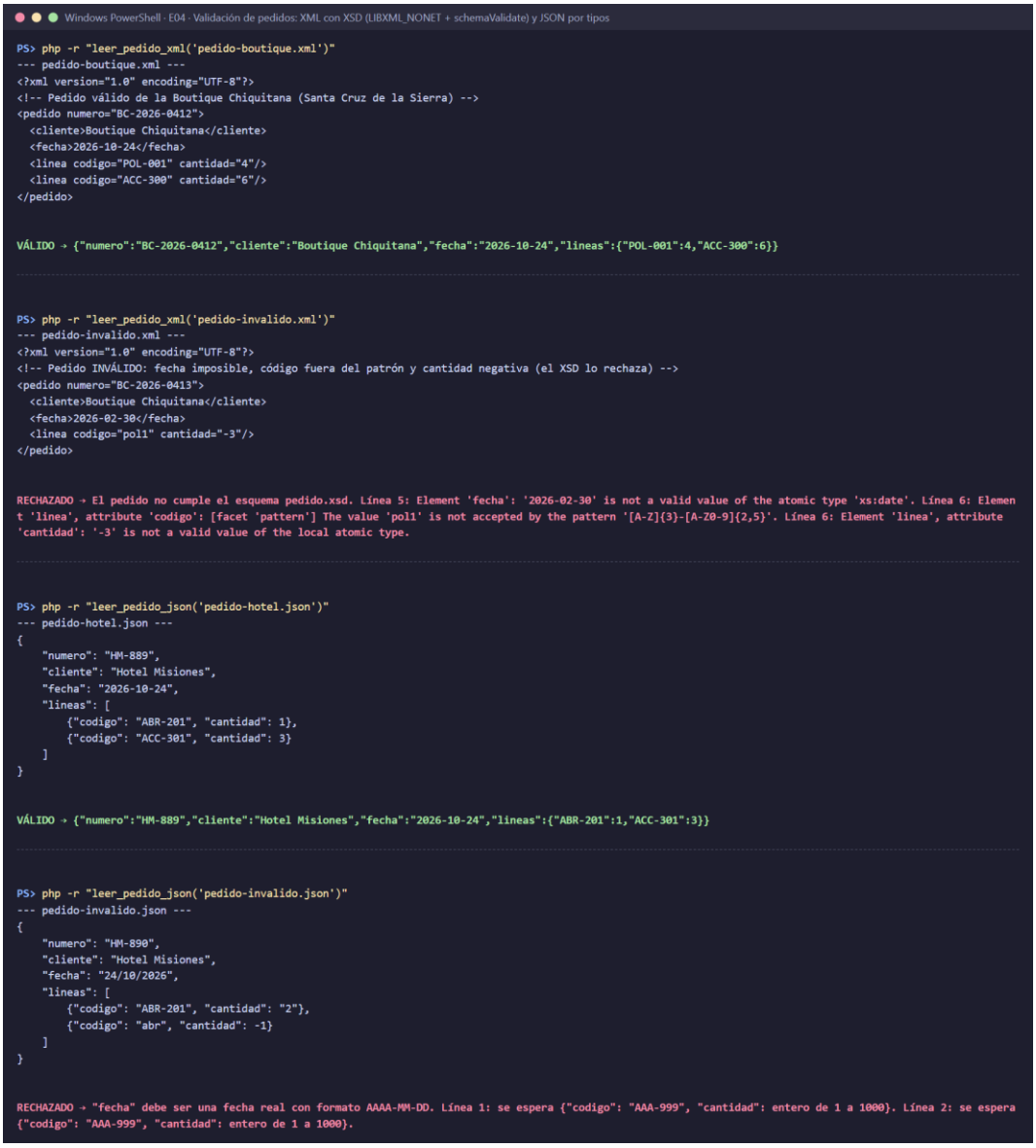
Tabla 2

Pedidos de prueba y por qué se rechaza cada dato

Pedido	Dato	Resultado	Motivo
pedido-boutique.xml	BC-2026-0412 · 4 POL-001, 6 ACC-300	Válido	Cumple el XSD
pedido-invalido.xml	fecha 2026-02-30	Rechazado	No es un xs:date válido
pedido-invalido.xml	codigo="pol1"	Rechazado	No cumple el patrón [A-Z]{3}-[A-Z0-9]{2,5}
pedido-invalido.xml	cantidad="-3"	Rechazado	No es xs:positiveInteger
pedido-hotel.json	HM-889 · 1 ABR-201, 3 ACC-301	Válido	Tipos y rangos correctos
pedido-invalido.json	"fecha": "24/10/2026"	Rechazado	Formato AAAA-MM-DD
pedido-invalido.json	"cantidad": "2" y -1	Rechazado	Debe ser entero entre 1 y 1000

Figura 4

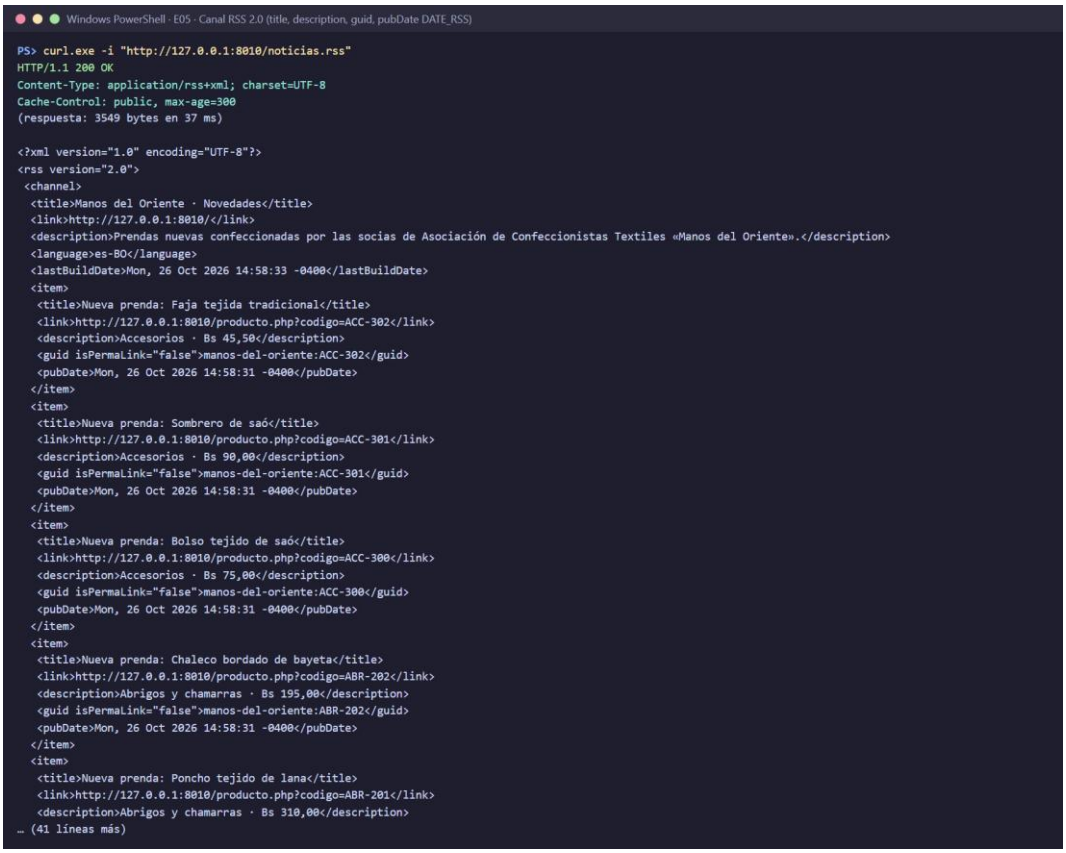
Validación de pedidos correctos e incorrectos (E04)



RSS y comparación SOAP/REST

Figura 5

Canal RSS 2.0 con guid y pubDate (E05)



El script `herramientas/soap_cliente.php` consulta la prenda POL-001 por SOAP y por REST en el mismo equipo. Ambos devuelven los mismos datos; la diferencia está en el formato (Tabla 3).

Tabla 3

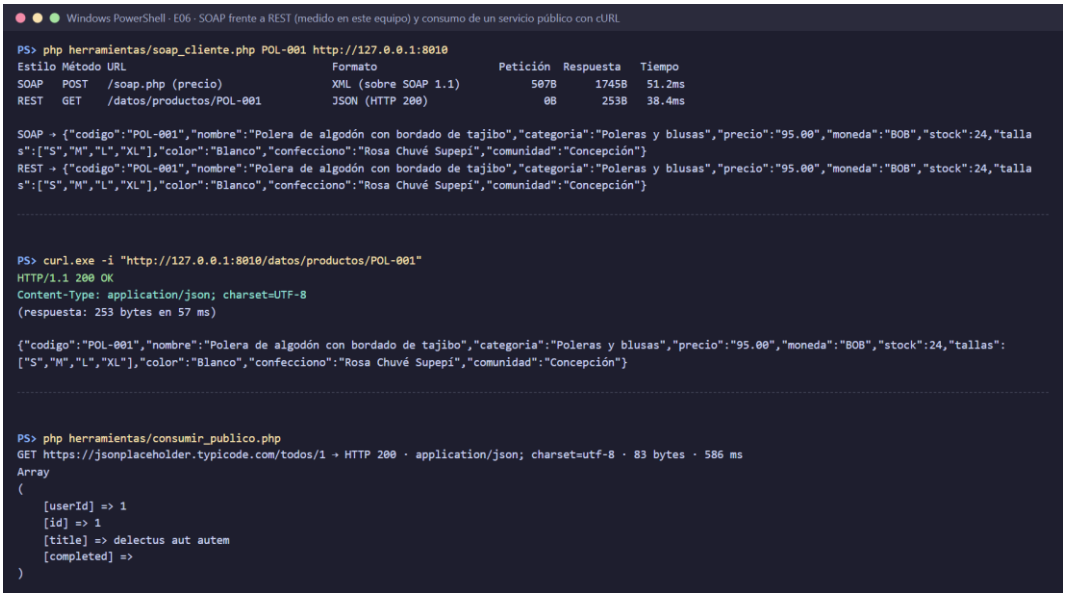
Comparación SOAP y REST medida en este equipo (26/10/2026)

Estilo	Método y URL	Formato	Petición	Respuesta	Tiempo
SOAP	POST /soap.php · precio("POL-001")	XML (sobre SOAP 1.1)	507 B	1.745 B	51 ms
REST	GET /datos/productos/POL-001	JSON	—	253 B	38 ms

Nota. Medición propia con PHP 8.4 y el servidor integrado; los tiempos varían entre ejecuciones.

Figura 6

SOAP frente a REST y consumo de un servicio público con cURL (E06)



Implementación del Laboratorio 13

`src/Api.php` atiende todo lo que llega a /api/v1: aplica CORS, responde OPTIONS, autentica el token, busca la ruta en una lista ordenada (la ruta fija /productos/sugerencias va **antes** de /productos/{codigo}) y convierte cualquier error en problem+json. Las escrituras reutilizan `validar_producto()`, la misma validación del formulario web.

Tabla 4

Contrato de la API /api/v1

Método	Ruta	Autenticación	Resultado
GET	/productos	Pública	200 datos, meta y enlaces · 422
GET	/productos/sugerencias?q=	Pública	200 · 422
GET	/productos/{codigo}	Pública	200 con ETag · 304 · 404
POST	/productos	Bearer (admin, operador)	201 + Location · 400 · 401 · 403 · 409 · 415 · 422
PUT	/productos/{codigo}	Bearer (admin, operador)	200 · 404 · 422
DELETE	/productos/{codigo}	Bearer (solo admin)	204 · 403 · 409
OPTIONS	/productos	—	204 preflight CORS
GET	/socios/{id}/liquidacion	Bearer (la socia, solo la suya)	200 · 403
GET, POST	/pedidos	Bearer (admin, operador)	200 · 201 con reserva
GET	/openapi.yaml	Pública	Contrato OpenAPI 3.1

Nota. Fichas completas por endpoint en docs/actividad-04/endpoints.md.

Tokens y seguridad

Los tokens tienen la forma `mdo_` + 40 hexadecimales. Se emiten en /tokens.php (solo administración), se muestran una única vez y en la base se guarda solo su hash SHA-256; cada token actúa con el rol de su dueño y puede revocarse. La API no usa cookies, por eso no necesita token CSRF. Ningún token aparece en el repositorio ni en las evidencias.

Figura 7

Lista paginada con datos, meta y enlaces (E07)



Figura 8

Creación con token: 201 Created y Location (E08)



Figura 9

Reemplazo con PUT: 200 (E09)



Figura 10

DELETE: 403 para el operador, 204 para administración y 404 después (E10)



Figura 11

Errores en problem+json: 401, 401, 403, 400, 415, 422, 422, 405, 409 y 404 (E11)



```
Windows PowerShell · E11 · Errores en application/problem+json: 400, 401, 403, 404, 405, 409, 415 y 422

PS> curl.exe -i -X POST "http://127.0.0.1:8010/api/v1/productos" --data-binary "@producto.json"
HTTP/1.1 401 Unauthorized
Cache-Control: no-store
WWW-Authenticate: Bearer realm="manos-del-oriente"
Content-Type: application/problem+json; charset=UTF-8
(respuesta: 296 bytes en 30 ms)

{
    "type": "https://github.com/TecnologiaWeb2/Tecnologiaweb2/blob/main/docs/actividad-04/endpoints.md#no-autenticado",
    "title": "No autenticado",
    "status": 401,
    "detail": "Esta operación requiere un token: \"Authorization: Bearer mdo_…\".",
    "instance": "/api/v1/productos"
}


PS> curl.exe -i -X POST "http://127.0.0.1:8010/api/v1/productos" -H "Authorization: Bearer $TOKEN" --data-binary "@producto.json"   # $TOKEN = token inventado
HTTP/1.1 401 Unauthorized
Cache-Control: no-store
WWW-Authenticate: Bearer realm="manos-del-oriente", error="invalid_token"
Content-Type: application/problem+json; charset=UTF-8
(respuesta: 274 bytes en 27 ms)

{
    "type": "https://github.com/TecnologiaWeb2/Tecnologiaweb2/blob/main/docs/actividad-04/endpoints.md#no-autenticado",
    "title": "No autenticado",
    "status": 401,
    "detail": "El token no es válido, venció o fue revocado.",
    "instance": "/api/v1/productos"
}


PS> curl.exe -i -X POST "http://127.0.0.1:8010/api/v1/productos" -H "Authorization: Bearer $TOKEN" --data-binary "@producto.json"   # $TOKEN = token de socio (o
culto)
HTTP/1.1 403 Forbidden
Cache-Control: no-store
Content-Type: application/problem+json; charset=UTF-8
(respuesta: 265 bytes en 37 ms)

{
    "type": "https://github.com/TecnologiaWeb2/Tecnologiaweb2/blob/main/docs/actividad-04/endpoints.md#prohibido",
    "title": "Prohibido",
    "status": 403,
    "detail": "El rol Socia o socio no permite esta operación.",
    "instance": "/api/v1/productos"
}


PS> curl.exe -i -X POST "http://127.0.0.1:8010/api/v1/productos" -H "Authorization: Bearer $TOKEN" --data-binary '{"codigo": "POL-051",'   # $TOKEN = token de o
perador (oculto)
HTTP/1.1 400 Bad Request
Cache-Control: no-store
Content-Type: application/problem+json; charset=UTF-8
(respuesta: 283 bytes en 61 ms)

{
    "type": "https://github.com/TecnologiaWeb2/Tecnologiaweb2/blob/main/docs/actividad-04/endpoints.md#peticion-mal-formada",
    "title": "Petición mal formada",
    "status": 400,
    "detail": "El cuerpo no es JSON válido: Syntax error.",
    "instance": "/api/v1/productos"
}


PS> curl.exe -i -X POST "http://127.0.0.1:8010/api/v1/productos" -H "Authorization: Bearer $TOKEN" -H "Content-Type: text/plain" --data-binary '{"codigo":"POL-0
51"}'   # $TOKEN = token de operador (oculto)
HTTP/1.1 415 Unsupported Media Type
Cache-Control: no-store
Content-Type: application/problem+json; charset=UTF-8
(respuesta: 315 bytes en 60 ms)

{
    "type": "https://github.com/TecnologiaWeb2/Tecnologiaweb2/blob/main/docs/actividad-04/endpoints.md#tipo-de-contenido-no-soportado",
    "title": "Tipo de contenido no soportado",
    "status": 415,
    "detail": "Envía el cuerpo con \"Content-Type: application/json\".",
    "instance": "/api/v1/productos"
}


PS> curl.exe -i -X POST "http://127.0.0.1:8010/api/v1/productos" -H "Authorization: Bearer $TOKEN" --data-binary '{"codigo":"xx","nombre":"A","categoria":"zapat
os","socio_id":6,"precio":"-1"}'   # $TOKEN = token de operador (oculto)
HTTP/1.1 422 Unknown Status Code
Cache-Control: no-store
Content-Type: application/problem+json; charset=UTF-8
(respuesta: 648 bytes en 56 ms)

{
    "type": "https://github.com/TecnologiaWeb2/Tecnologiaweb2/blob/main/docs/actividad-04/endpoints.md#datos-invalidos",
… (82 líneas más en E11_errores_401_403_422.txt)
```

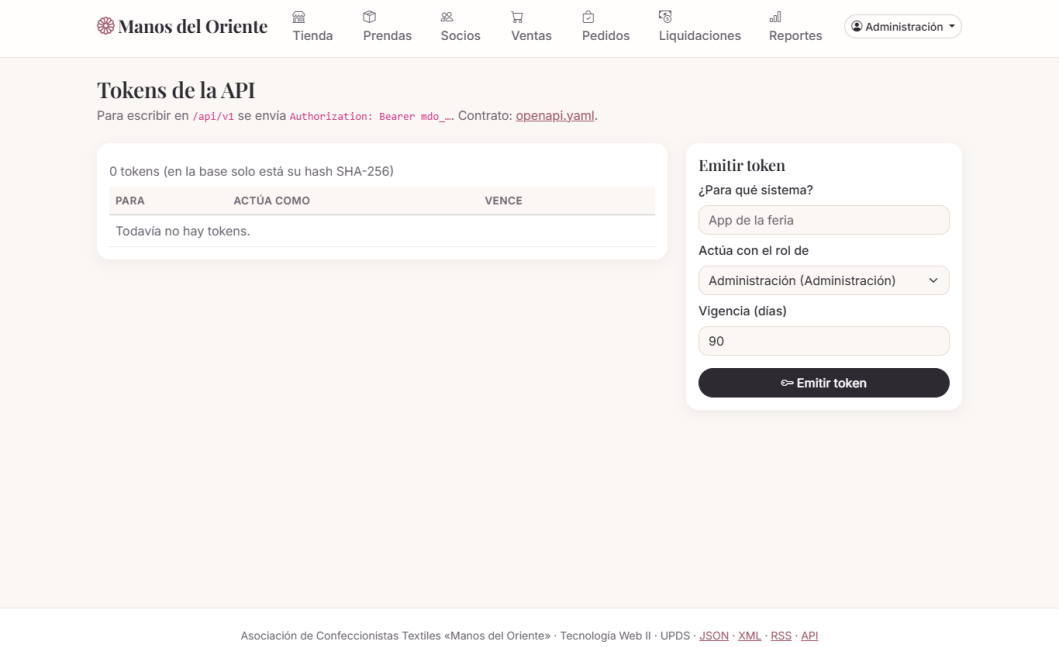
Figura 12

ETag y 304, OPTIONS con CORS, OpenAPI 3.1 y tokens guardados como SHA-256 (E12)



Figura 13

Pantalla de tokens de la API (E12)



Implementación del Laboratorio 14

La vitrina conserva su versión dibujada en el servidor (funciona sin JavaScript) y, encima, `public/js/catalogo.js` la actualiza con la API. `api.js` centraliza fetch: revisa response.ok, lee el problem+json, distingue un fallo de red de una cancelación (AbortError) y muestra el aviso en un contenedor role="alert". Cada carga cancela la anterior y la búsqueda espera 250 ms sin escribir antes de pedir. Los datos se dibujan con createElement, textContent y replaceChildren; también `app.js`, que antes usaba innerHTML para los modales, ahora analiza el HTML propio con DOMParser.

Figura 14

Grilla actualizada con fetch al filtrar por categoría (E13)

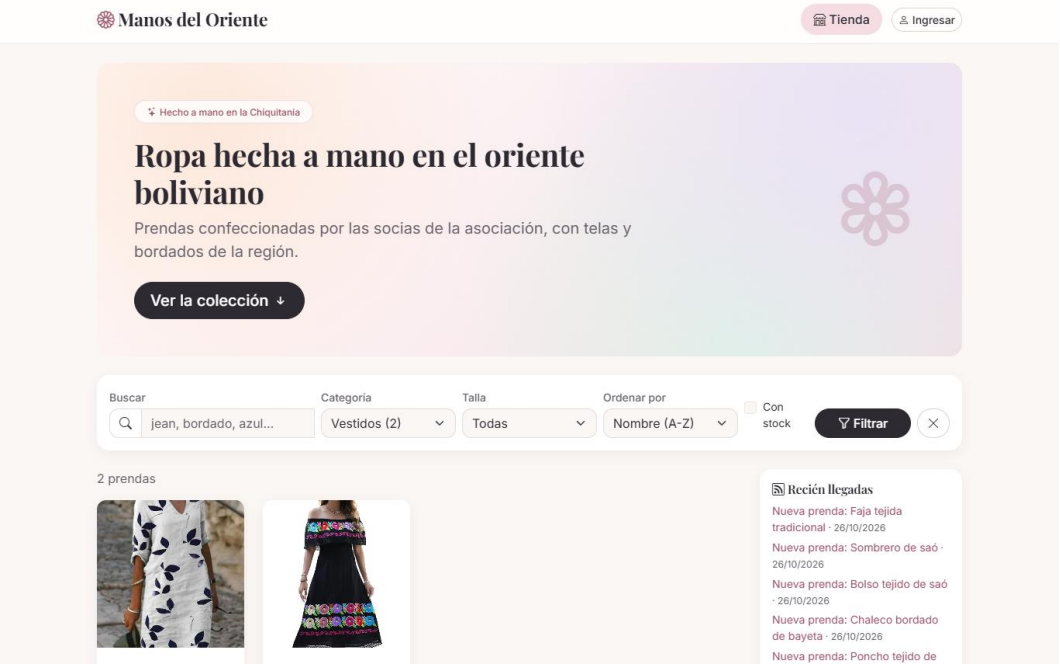


Figura 15

Orden por precio y paginación sin recargar la página (E14)

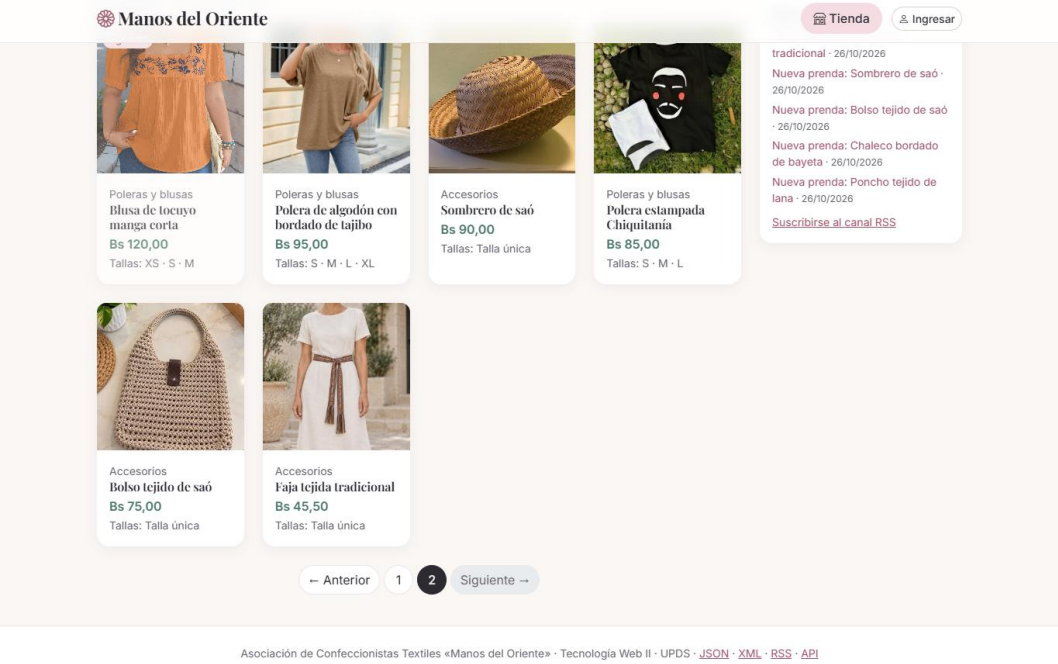


Figura 16

Autocompletar con role=combobox, listbox y opción activa (E15)

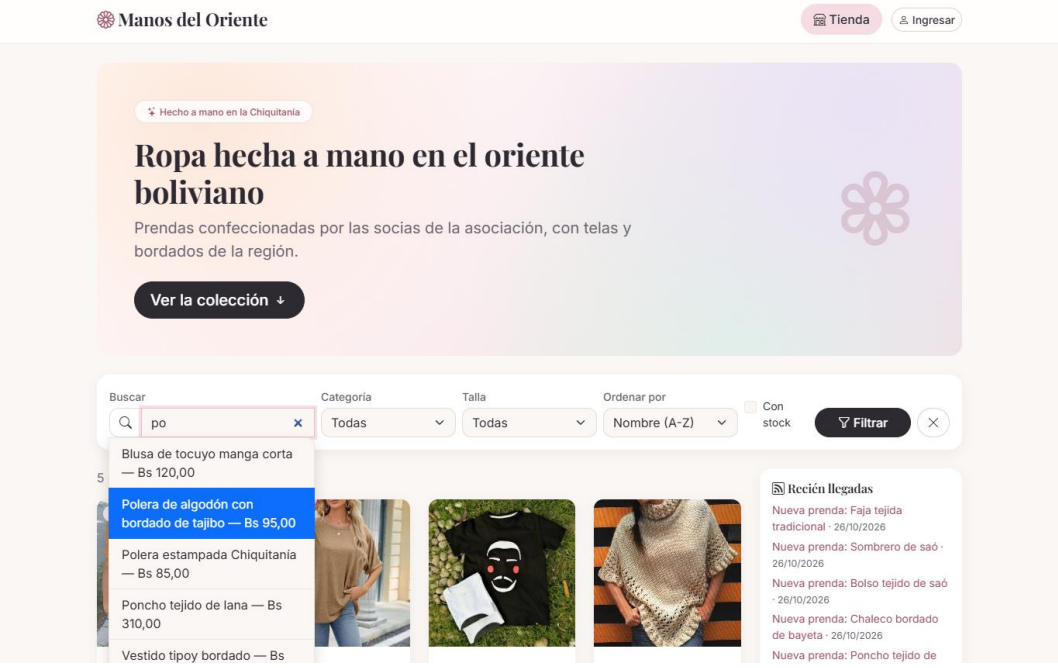
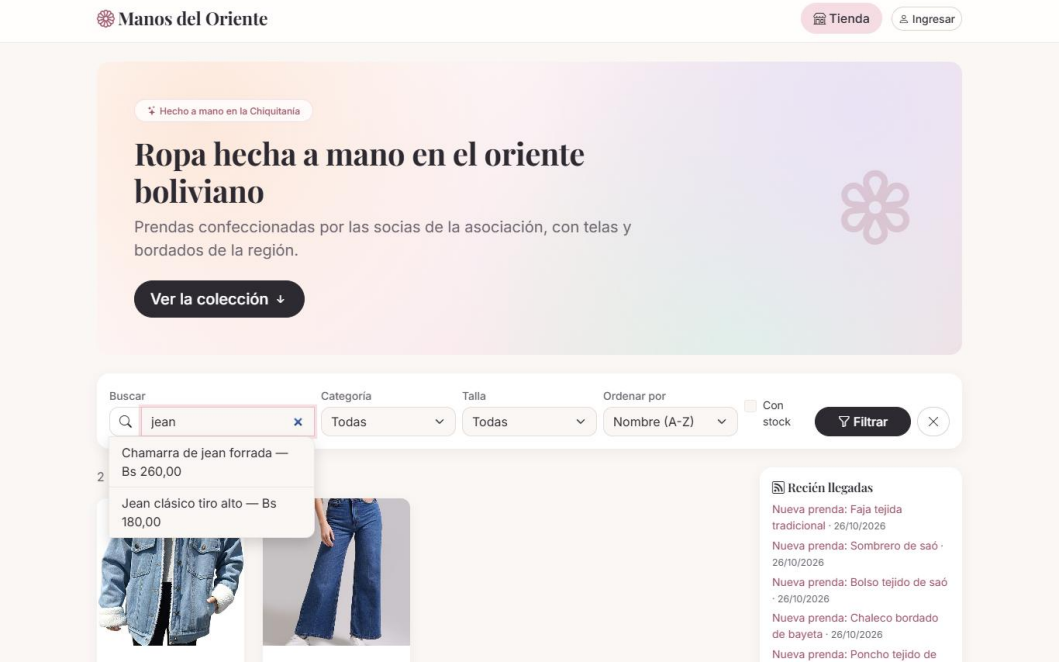


Figura 17

Con red lenta, las búsquedas «bo» y «bord» se cancelan y gana «jean» (E16)



Nota. La prueba automática midió 3 peticiones canceladas.

Figura 18

Validación en vivo: POL-001 ya existe (200) (E17)

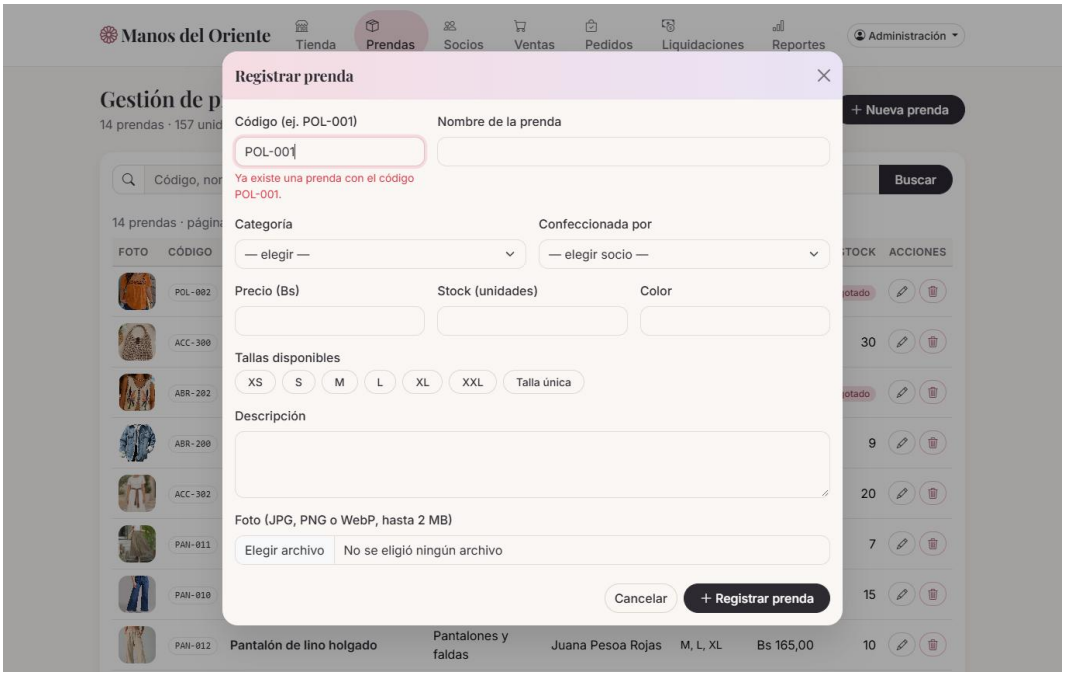


Figura 19

Validación en vivo: POL-999 está disponible (404) (E17)

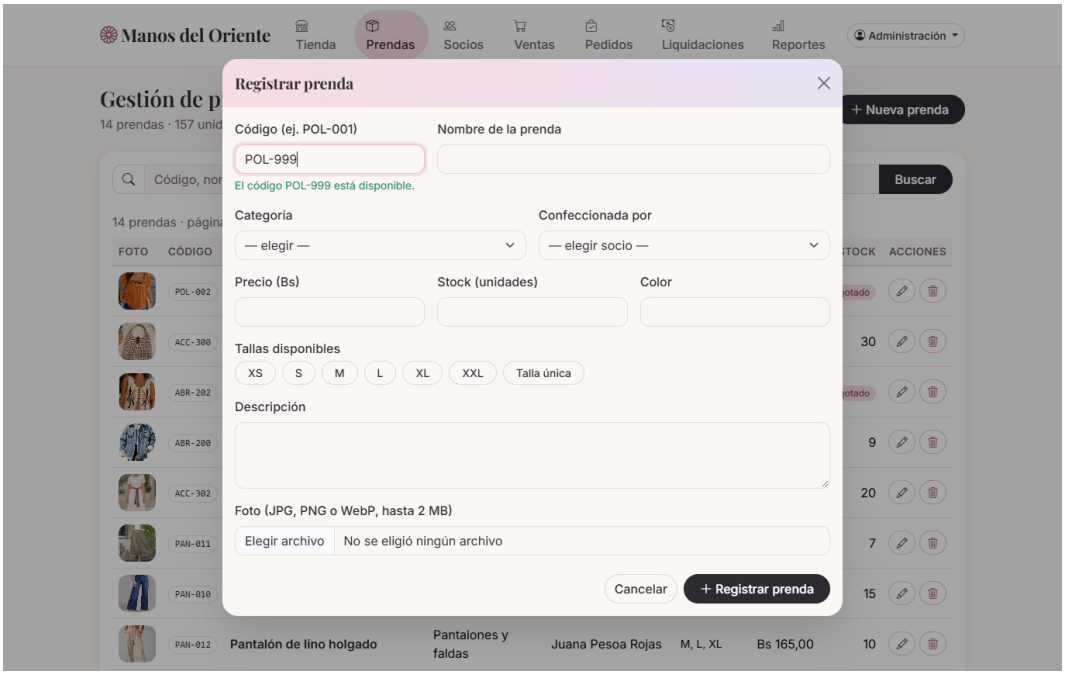
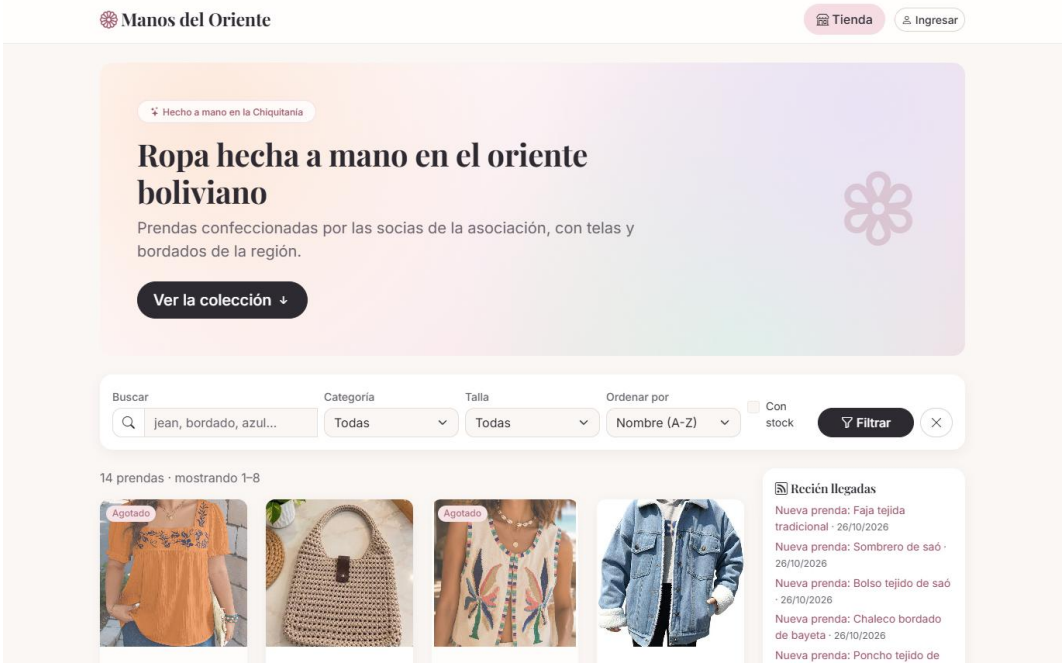


Figura 20

Cinco novedades del RSS leídas con DOMParser (E18)



Pruebas y evidencias

Tabla 5

Matriz de evidencias

Código	Evidencia	Estado	Figura o archivo
E01	Versiones y entorno	✓	Figura 1
E02	Catálogo JSON	✓	Figura 2
E03	XML y escape	✓	Figura 3
E04	Validación XSD/JSON	✓	Figura 4
E05	RSS	✓	Figura 5
E06	SOAP/REST y cURL	✓	Figura 6
E07	GET paginado	✓	Figura 7
E08	POST 201	✓	Figura 8
E09	PUT 200	✓	Figura 9
E10	DELETE 204	✓	Figura 10
E11	Errores 401/403/422	✓	Figura 11
E12	OpenAPI/ETag/CORS	✓	Figuras 12 y 13
E13	Grilla AJAX	✓	Figura 14
E14	Paginación y orden	✓	Figura 15
E15	Autocompletar accesible	✓	Figura 16
E16	Cancelaciones	✓	Figura 17
E17	Validación en vivo	✓	Figuras 18 y 19
E18	RSS en interfaz	✓	Figura 20
E19	Verificadores finales	✓	Figuras 21 y 22
E20	GitHub e informe	✓	https://github.com/TecnologiaWeb2/Tecnologiaweb2

`php verificar.php` ejecuta 136 comprobaciones: 59 de la actividad 2 (las 57 originales y 2 nuevas), 27 de la actividad 3, 9 del laboratorio 12, 15 del 13, 7 del 14 y 19 de los módulos del trabajo final y de la seguridad del código. Pasaron todas en MariaDB 10.11 y en MySQL 8.4. Un segundo script controla Edge y comprueba 12 comportamientos de la interfaz.

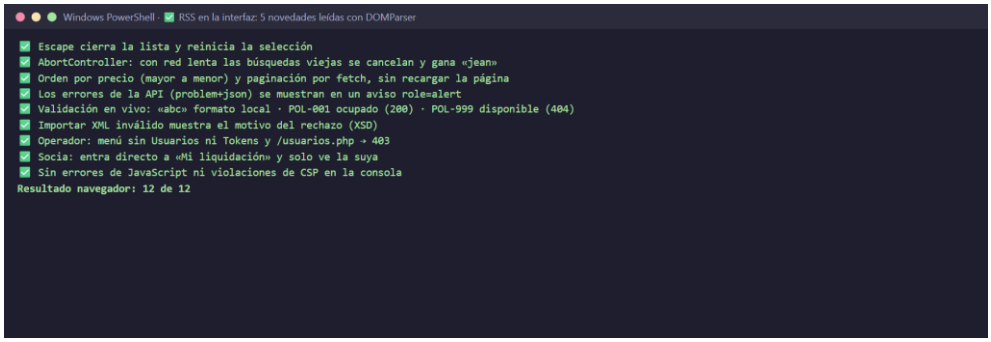
Figura 21

php verificar.php: 136 de 136 (E19)



Figura 22

Pruebas en el navegador: 12 de 12 (E19)



Bitácora de IA

Tabla 6

Resumen de la bitácora de IA de la actividad

Fecha · Lab	Pedido	Propuesta de la IA	Decisión propia	Prueba
26/10 · 12	Generar XML seguro	XMLWriter en lugar de concatenar; LIBXML_NONET para leer	Aceptado; se agregó la prueba de XXE	Escape & y XXE
26/10 · 12	Pedidos JSON	Validar tipos con is_int	Aceptado: «"2"» se rechaza	pedido-invalido.json
26/10 · 13	Manejar errores HTTP	problem+json para todo, con «errores» en 422	Ajustado: 409 para código repetido	E11
26/10 · 13	Reporte con fechas	Repetía :desde en una consulta	Corregido: con preparadas reales no se puede	Lectura del código
26/10 · 14	Evitar carreras de fetch	AbortController + debounce	Aceptado; app.js sin innerHTML	E16 (3 canceladas)
26/10 · 14	Revisión visual	Capturas con Edge	Las fotos estaban cruzadas: se reasignaron por hash MD5	E13–E15

Nota. Detalle completo y reflexiones de cada integrante en docs/BITACORA_IA.md.

Resultados y discusión

Se cumplieron los tres laboratorios dentro del sistema propio, no solo en las carpetas del docente. La medición de la Tabla 3 confirma lo esperado: el sobre SOAP hace que la misma prenda pese casi siete veces más que en JSON, y REST se prueba con un navegador o curl; SOAP conserva la ventaja de un contrato formal cuando se publica un WSDL, que aquí no se usó. ETag reduce el tráfico de las consultas repetidas: la segunda petición de la Figura 12 no tiene cuerpo.

Las pruebas encontraron errores que una revisión a ojo no había visto: una consulta que repetía un parámetro nombrado (no se admite con consultas preparadas reales), un identificador HTML duplicado y fotos de demostración asignadas a la prenda equivocada. También mostraron un límite del hosting gratuito elegido para publicar: InfinityFree pide un desafío JavaScript a cada visitante, de modo que la API se consume sin problemas desde el navegador, pero un cliente como curl necesita la cookie de ese desafío.

Una decisión de diseño fue mantener la vitrina dibujada en el servidor y mejorarla con fetch, en lugar de una página vacía que dependa del JavaScript: los buscadores y quien tenga JavaScript desactivado siguen viendo el catálogo.

Conclusiones

- El catálogo se puede consumir en JSON, XML, RSS, SOAP y REST con una sola representación de los datos, sin exponer datos personales.
- Validar antes de usar (XSD para XML, tipos para JSON) evita que un pedido mal formado llegue a la base.
- La API cumple el contrato de la guía: códigos HTTP correctos, problem+json, tokens con hash, ETag, CORS y OpenAPI 3.1.
- fetch exige revisar response.ok y cancelar peticiones viejas; con AbortController la última búsqueda siempre prevalece.
- Las pruebas automáticas (136 + 12) y las evidencias generadas con peticiones reales hacen que los resultados sean verificables.

Referencias

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. https://doi.org/10.17487/RFC9110

International Organization for Standardization. (2019). *Date and time — Representations for information interchange — Part 1: Basic rules* (ISO 8601-1:2019).

Jones, M., & Hardt, D. (2012). *The OAuth 2.0 authorization framework: Bearer token usage* (RFC 6750). Internet Engineering Task Force. https://doi.org/10.17487/RFC6750

MDN Web Docs. (s. f.-a). *AbortController*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/AbortController

MDN Web Docs. (s. f.-b). *Uso de Fetch*. Mozilla. https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

Nottingham, M., Wilde, E., & Dalal, S. (2023). *Problem details for HTTP APIs* (RFC 9457). Internet Engineering Task Force. https://doi.org/10.17487/RFC9457

OpenAPI Initiative. (2021). *OpenAPI Specification v3.1.0*. https://spec.openapis.org/oas/v3.1.0

OWASP Foundation. (s. f.). *XML External Entity prevention cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html

Requena Llorentty, J. N. (2026). *Guía integral del estudiante: Actividad 4, Tecnología Web II* [Material de clase]. Universidad Privada Domingo Savio.

RSS Advisory Board. (2009). *RSS 2.0 specification*. https://www.rssboard.org/rss-specification

The PHP Group. (s. f.). *XMLWriter*. Manual de PHP. https://www.php.net/manual/es/book.xmlwriter.php

W3C Web Accessibility Initiative. (s. f.). *Combobox pattern*. ARIA Authoring Practices Guide. https://www.w3.org/WAI/ARIA/apg/patterns/combobox/

World Wide Web Consortium. (2007). *SOAP version 1.2 part 1: Messaging framework* (2.ª ed.). https://www.w3.org/TR/soap12-part1/

World Wide Web Consortium. (2008). *Extensible Markup Language (XML) 1.0* (5.ª ed.). https://www.w3.org/TR/xml/

World Wide Web Consortium. (2012). *W3C XML Schema Definition Language (XSD) 1.1 part 1: Structures*. https://www.w3.org/TR/xmlschema11-1/

Anexos

Anexo A. Repositorio y archivos de la actividad

Tabla 7

Ubicación de los entregables en el repositorio

Recurso	Ubicación
Repositorio	https://github.com/TecnologiaWeb2/Tecnologiaweb2
Documentación de endpoints	docs/actividad-04/endpoints.md
Contrato OpenAPI 3.1	docs/openapi.yaml
Evidencias E01–E20	docs/actividad-04/evidencias/
Laboratorios 12, 13 y 14	laboratorios/lab12 … lab14
Pruebas	verificar.php y pruebas/actividad4.php
Bitácora de IA	docs/BITACORA_IA.md

Anexo B. Cómo reproducir los resultados

```
php bd/instalar.php
php -S 127.0.0.1:8000 -t public public/enrutador.php
php verificar.php                                    # 136 de 136
php herramientas/evidencias_api.php http://127.0.0.1:8000
php herramientas/soap_cliente.php POL-001
php herramientas/consumir_publico.php
```